

## Future Thoughts: An AI-Native Workflow System

1. The language
2. Plans as data, stored and fetched dynamically
3. Handlers
4. Distributed execution
5. Human verification mode
6. UI interaction
7. Consequences and tradeoffs
8. Relationship to Facetwork today
9. Concrete first steps already taken

# Future Thoughts: An AI-Native Workflow System

---

*Forward-looking design exploration, companion to the Facetwork thesis.*

This document captures a design exploration that asks: *if the audience for a workflow language were AI agents rather than humans, and we weren't bound to Python / JVM / any particular host runtime, what would the language, handlers, distributed execution, and UI look like?*

Facetwork today is human-first — FFL is a readable DSL, handlers are Python modules, the dashboard is a human's window into runs. The exercise below inverts that: humans become verifiers, not authors; agents become authors, not just consumers. It is a design exploration, not an implementation plan. No code changes are proposed.

---

## 1. The language

---

Human languages optimize for **readability, stability, and cognitive chunking**: whitespace, keywords, scoped names, familiar control flow. An agent doesn't need any of that. Agents need:

- **Structural unambiguity.** No parser error recovery, no "did you mean," no context-sensitive indentation. A graph or tree is enough. The "source" should be a canonical serialized IR, not a DSL.
- **Content-addressed immutability.** Every node (step, schema, expression) is identified by a hash of its definition. An agent proposing "the same step as run #4729 but with `region=bavaria`" can express that by reference + delta instead of re-emitting text. Plans become cheap to compose, diff, and cache.
- **Typed holes and refinement.** An agent often knows *shape* before *content*: "a step that produces `RouteFeatures`, implementation TBD." The language should have first-class holes with type constraints, and execution should be able to pause at a hole, ask a planner to fill it, and resume.
- **Effects in the type.** "This step reads PostGIS," "this step spends tokens," "this step is non-idempotent," "this step costs money" — encoded as effect rows, not discovered from docs. The scheduler and the human verifier both read effects.
- **No sugar.** Mixins, `andThen`, `implicit`, `catch when` — all great for humans, all noise for agents. Desugar everything to: *node, dependency, effect, schema*.

So the "language" is really two things: a **canonical IR** (what executes) and a **projection layer** (what a human sees when verifying). FFL-style surface syntax becomes one of many projections, alongside a graph view, a natural-language paraphrase, and a diff-against-prior-run view.

## 2. Plans as data, stored and fetched dynamically

---

Agents don't ship code; they **write rows**. A plan is a set of content-addressed nodes in a store (Mongo, Postgres, a Merkle store — doesn't matter). Consequences:

- An agent composes a plan by `INSERT ... ON CONFLICT DO NOTHING` over node hashes. Two agents proposing the same subgraph converge for free.
- A library of **reusable subgraphs** emerges naturally — no package manager, no import paths, just hashes. "Use the OSM-extract-and-render subgraph from run #4729" is a single reference.
- Execution is: walk the DAG, for each ready node look up handler-by-hash, dispatch. The plan can be extended mid-run by an agent appending new nodes downstream of a completed one, as long as dependencies point backward. This is how you get "planned actions known at plan time, exact mechanics at runtime."
- **Speculative branches** are first-class: an agent can emit a plan with three alternative subtrees and a `choose-at-runtime` node whose handler is "ask the planner, given these intermediate results." This subsumes `catch when`, A/B tests, and human approval gates into one primitive.

## 3. Handlers

---

Today a handler is "a Python function on a runner." For an agent-native system:

- **Handlers are capabilities, not code.** A handler is declared by the effects it provides and the schemas it consumes/produces. The implementation may be a Python module, a container image, a shell command, a SQL query, an HTTP call, a prompt to another model, or a subgraph of other handlers. The runtime picks an implementation at dispatch time based on cost, latency, availability, and trust.
- **Handler selection is itself a scheduled decision.** "Extract routes from PBF" could be served by osmium locally (fast, cheap, trusted) or by a Claude prompt (slow, expensive, flexible). The scheduler picks; the plan doesn't hardcode.
- **Sandboxing by effect, not by language.** A handler declaring `reads: postgis` gets a read-only connection injected; one declaring `spends: tokens<=5000` gets a token-budgeted client. The runtime enforces; the handler doesn't have to be careful.
- **Idempotency as a type.** Non-idempotent handlers are opt-in and must declare a compensation. The runtime refuses to auto-retry anything not marked idempotent — no more "did that retry double-charge the user?"

## 4. Distributed execution

---

Most of Facetwork's runtime is already aligned with what's needed — leases, heartbeats, reapers, drain, repair. The agent-native shifts are:

- **The scheduler is a first-class agent, not a thread pool.** It reads the plan, open holes, current resource state, budget, and deadlines, and emits dispatch decisions. It can itself be paused, resumed, and replaced mid-run.
- **Every state transition is an append to a log,** and the log is the source of truth. Current execution state is a fold over the log. This makes "re-run from here," time-travel debugging, and cross-run analysis uniform — they're all queries against the log.
- **Resource budgets are explicit and hierarchical.** A plan declares " $\leq \$5$ ,  $\leq 10\text{min}$ ,  $\leq 100\text{k}$  tokens,  $\leq 3$  retries per node." The scheduler enforces. Running out of budget pauses the plan and notifies the verifier. No surprise bills.
- **Cross-plan composition.** A step can *await* another plan's output by hash. This replaces "workflow triggers workflow" with a single substrate.

## 5. Human verification mode

---

This is the load-bearing piece. If agents write plans and pick handlers at runtime, how does a human stay in control?

- **Verification is a projection over the plan + budget + effects,** not over code. The human sees: "this plan will read PostGIS (read-only), spend up to \$2 in Claude tokens, write 3 files under `/data/osm/bavaria/`, and take  $\sim 8$  minutes. Here's the natural-language paraphrase. Here's the graph. Here's the diff against the last approved plan of this shape."
- **Approve at the level of effects and budgets,** not at the level of nodes. "I approve any plan that reads-only from PostGIS, spends  $\leq \$2$ , and writes only under `/data/osm/`" becomes a reusable policy. The scheduler checks plans against policies; only policy-violating plans escalate.
- **Runtime-known mechanics are fine if the effects are bounded.** The human doesn't need to know *which* SQL query will be generated, only that whatever is generated will be read-only and will time out at 30s. This is the key unlock: the human verifies **envelopes**, the agent fills in **contents**.
- **Pause points are declared in the plan,** not bolted on. A node can declare "requires human confirmation before dispatch" with a natural-language summary. The verifier sees a queue of such confirmations, not a firehose of every step.
- **Explain-then-act is the default for novel plans.** First run of a new plan shape: dry-run, show effects, ask. Subsequent runs within policy: auto.

## 6. UI interaction

---

The dashboard today is step-centric because humans author step-by-step. For an agent-native system:

- **Primary view: the plan, not the run.** A graph/outline of what's going to happen, with effect and budget annotations, and a paraphrase. Runs are instances of plans; the plan is the noun.
- **Secondary view: the policy inbox.** Plans awaiting approval, plans that broke budget, plans that hit a declared pause point. Everything else runs without interrupting.
- **Tertiary view: the log explorer.** Queries over the append-only log — "show me all plans that touched this table in the last week," "diff this run's effects against the last approved run of the same shape."
- **No "New Run" button for humans.** Humans write *intents* ("geocode Bavaria, render amenities"); a planning agent proposes a plan; the human verifies the envelope; execution starts.

## 7. Consequences and tradeoffs

---

- **Debuggability shifts.** "Read the code" stops being the first move. "Read the plan, read the log, read the policy" replaces it. Tooling has to make those as fluent as a stack trace.
- **Trust boundary moves up.** You no longer trust handlers to be well-behaved — you trust the effect system and the sandbox. Bugs in the effect system are catastrophic in a way that bugs in a single Python handler are not.
- **Human skill shifts.** Authoring FFL → writing policies, reviewing envelopes, debugging plans. This is closer to an SRE / compliance role than to a programmer role.
- **Dead weight to drop.** Surface syntax sugar, handler discovery by import path, per-handler retry configuration, most of the dashboard's step-level affordances. All replaced by plan + policy + log.
- **The biggest risk** is the effect system being wrong or under-specified. If "reads PostGIS" doesn't actually constrain what a handler does, the whole verification story collapses. This is where most of the engineering goes.

## 8. Relationship to Facetwork today

---

Most of the existing runtime infrastructure survives: the task/lease/heartbeat substrate, the runner fleet, the step log, the repair machinery. What changes is the layer above:

- FFL stays as one projection (the human-readable one), but the IR becomes canonical and content-addressed.
- Handlers gain effect declarations and become capability-typed.
- The scheduler becomes pluggable and agent-driven.
- The dashboard grows a plan view, a policy inbox, and a log explorer; the step-detail page becomes a debugging tool, not the primary UI.

A plausible incremental path: add effect annotations to existing event facets → add a policy language → add a content-addressed IR alongside FFL → add a planning agent that emits IR → move the dashboard's primary view to plans. Each step is useful standalone; the endpoint is the agent-native system.

## 9. Concrete first steps already taken

---

Three pieces of recent work line up with the speculation above closely enough to be worth flagging. None of them are the agent-native endpoint; each is a first step that the design exploration above would, in retrospect, have predicted.

**Standalone packages as proto-sealed-skills (§4).** Eight `fwh_*` repositories now exist as pip-installable Facetwork example packages — `fwh_osm`, `fwh_osm_lz`, `fwh_noaa_weather`, `fwh_jenkins`, `fwh_census_us`, `fwh_genomics`, `fwh_sensor_monitoring`, and `fwh_anthropic`. Each ships its own FFL sources, its own handlers, and its own pinned dependencies; each declares a `facetwork.examples` entry point that Facetwork runners discover at start time. Two pieces are missing relative to a true sealed skill: the content-addressable identifier (the unit today is a Git commit, not a hash of the FFL + handler set), and the formal provenance metadata (model, prompt, reviewer, tests). Both are additive — a sealing tool that produced `(content_hash, fingerprint, manifest)` from an `fwh_*` directory would convert the existing packages into sealed skills without changing their internal structure. The substrate is the FFL + entry-point pair; sealing is a layer on top.

**Vendor wrappers as effect-typed carriers (§3).** The `fwh_anthropic` package wraps six Anthropic vendor surfaces (Messages, Batch, Files, Agent SDK, Claude Code, Computer Use) as 16 typed event facets. The effects are well-known per surface — Messages calls cost tokens; Batch calls cost tokens at half-price but with 24-hour latency; Files calls hit Anthropic's storage; Claude Code runs a subprocess; Computer Use takes control of a display. Today those effects live in prose; the *facets themselves* are typed at the parameter and return level. The minimal next step is the one §3 names: lift the effects into the type signature. A facet declared `=> (result: MessageResult) effects (anthropic.tokens, anthropic.cache_read)` would let a future scheduler pick between this facet and a cheaper one based on declared cost, exactly as §4 imagines, without changing any call site. Vendor wrappers are the highest-leverage place to start because their effect signatures are knowable and externally stable.

**Effect/cost annotations + capability discovery (§3, and the §8 path's first step).** The §8 incremental path opens with "add effect annotations to existing event facets"; that step is now taken. Facets carry effect and cost through the existing mixin grammar — with `Effect(kind = "pure"|"external"|"io")`, with `Cost(tier = "free".."expensive")`, no new syntax — and a capability index (`facetwork/capabilities/`, the `fw_capabilities` MCP tool) reads them, so an authoring agent can query the library by intent and filter to pure, cheap primitives or see which steps hit an engine. All 247 event facets of the `fwh_osm` package are annotated. This is exactly the move the vendor-wrapper paragraph above calls the minimal next step — "today those effects live in prose" — taken in a deliberately conservative form: the effects are *discovery* metadata an authoring agent reads, not yet a typed effect row a scheduler dispatches on. The remaining gap is the one §3 names — lifting with `Effect(...)` from a side annotation into the facet's type signature, `=> (...) effects (...)`, and giving a scheduler the policy to act on it. But the precondition that vision needs — a machine-readable, per-facet statement of what a step costs and whether it is side-effect-free, populated across a real library, not living in docstrings — is now in place. Cost-aware dispatch has something concrete to read; what it lacks is the type-level binding and the scheduler itself.

**Cross-package composition + JSON-bridge fields (§1, §2).** The `research-agent` example was ported to consume `anthropic.messages.CreateMessage` from the `fw_h_anthropic` package by qualified name, plus a small set of typed `Parse<Schema>` event facets local to its own namespace. From the FFL author's perspective, there is no marker that the LLM call lives in a different package; the discovery is operational, not linguistic. The `anthropic.compose.DocumentQA` workflow does the same in the other direction — a small composed workflow shipped as part of a vendor wrapper, available to any consumer that imports the namespace. Both are concrete demonstrations of "plans as data, stored and fetched dynamically" (§2), once you allow that *the unit of storage today is a package, not a content hash, and the fetch is `pip install`, not a Merkle traversal*. The transport mechanism is not yet what §2 envisages, but the property the transport delivers — qualified-name reference, late binding to implementation, composition without rewrite — is already in place. Tightening the transport into a content-addressed registry is mechanical compared to the algebra that makes the composition meaningful.

The same package also makes one weakness visible: nested-list payloads are carried through FFL today as JSON-encoded strings (`tools_json`, `messages_json`, `trace_json`, `results_json`, `files_json`) because the current type system does not natively model arbitrary lists of records as facet attributes. This is exactly the desugar-everything-to-`{node, dependency, effect, schema}` pressure §1 anticipated. Either FFL grows typed records-in-lists (a tractable addition that does not disturb the rest of the algebra), or the next-generation IR §1 sketches takes over as the canonical form and FFL becomes one projection of it. The first option is incremental and likely; the second is the agent-native endpoint and longer-horizon. Either way, the JSON-bridge fields are evidence that the language is already feeling the constraint the speculation anticipates.

None of this is the agent-native system. It is what the *human-first system* looks like when AI-author and cross-package distribution start to be load-bearing on its existing primitives. The interesting question, in light of this, is which §1–§8 ideas are reachable from here as additive layers (sealing, effect typing, plan registry) and which require the canonical IR + projection split §1 calls for. The first set is plausibly an incremental roadmap; the second set is, on this evidence, still a redesign rather than a refactor.

---

*This is a design exploration, not an implementation plan. The §9 observations are recent; the §1–§8 design exploration predates them.*